

STUDY GUIDE · FROM ZERO TO RUNNING IT YOURSELF

Automated Linux Server Hardening & Monitoring with Ansible — Study Guide

Every tool explained for beginners: Linux server basics, SSH and keys, Ansible, nftables, fail2ban, automatic updates, kernel hardening, auditd, Lynis, Prometheus and node_exporter, Grafana, Molecule and GitHub Actions, plus hands-on labs, a glossary and 25 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-ansible-server-baseline>

Contents

- 0. The big picture (no jargon)
- 1. Linux server basics
- 2. SSH and keys
- 3. Ansible
- 4. Firewall with nftables
- 5. fail2ban
- 6. Automatic security updates
- 7. Kernel hardening
- 8. auditd
- 9. Lynis
- 10. Prometheus and node_exporter
- 11. Grafana
- 12. Molecule and Docker
- 13. GitHub Actions
- 14. How everything fits together
- 15. Hands-on labs

Glossary

Interview questions this project prepares you for

Each chapter: what the tool is, why this project uses it, how it works, where it is integrated, commands to try, common mistakes, and questions with answers.

0. The big picture (no jargon)

The problem, as a story

A company rents a new Linux server. The provider hands it over with **default settings**: anyone on the internet can try to log in with a password, the administrator account `root` can log in directly, every network port that a program opens is reachable, and nobody checks whether security updates get installed.

Within minutes, automated scanners find it and start guessing passwords. This happens to every server with a public address.

The usual fix: an engineer logs in and changes settings by hand, following a checklist from a blog post. On the next server they forget a step. Six months later someone "temporarily" re-enables password login and never switches it off. Nobody notices, because nobody re-checks.

What this project does instead

Think of a company that owns many buildings. Instead of letting each caretaker secure their building their own way, it writes **one security checklist**: - only people with a key card get in, and there is no master key at the front door - every door is locked except the ones on a short list - anyone who tries the wrong key card five times is turned away for an hour - the locks are upgraded automatically when the manufacturer announces a weakness - a logbook records who changed the alarm settings - a **guard walks through every building each week** with the checklist and writes down a score

This project is that checklist, written as code (**Ansible**), plus the guard (**Lynis** weekly audits and **Prometheus** monitoring):

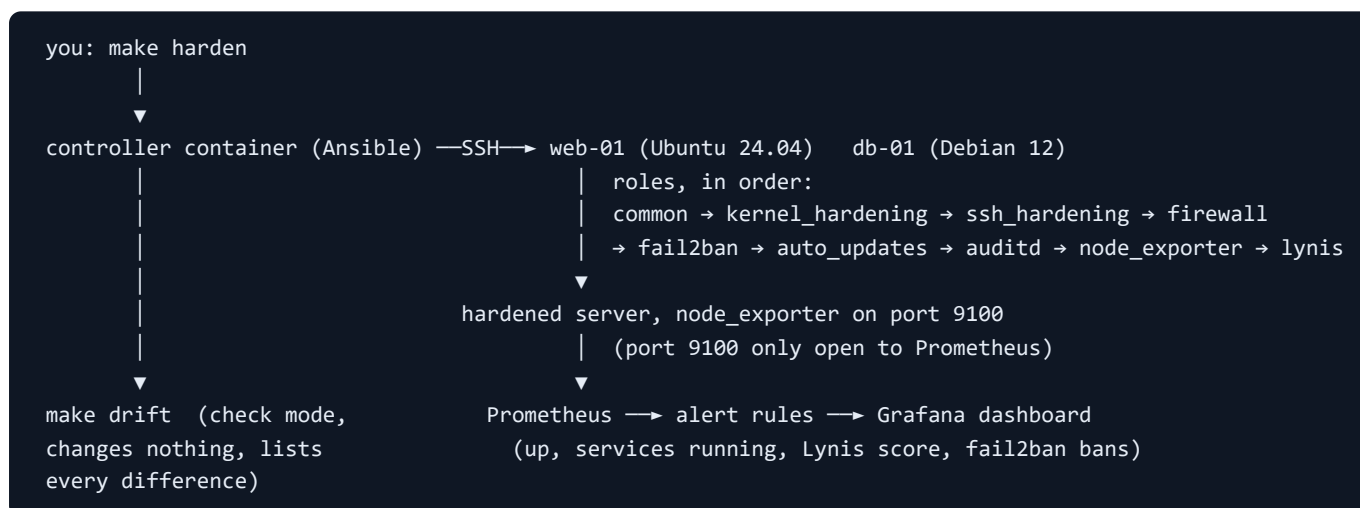
1. **One command** (`make harden`) applies the same checklist to every server.
2. **Running it again changes nothing** if everything is still correct. If someone changed a setting by hand, the **drift check** shows exactly what, and the next run puts it back.
3. A **security score** is measured before and after, so the improvement is a number, not an opinion.
4. **Dashboards and alerts** show if a protection stops running, if the score drops, or if someone is being banned for guessing passwords.

The pieces, and their jobs

Piece	Job in one sentence	Building analogy
Ansible	Applies the checklist to every server over SSH, and only changes what differs	The facilities team with the master checklist
SSH hardening	Key-only logins, no direct root login, modern encryption	Key cards only, no master key at the front door
nftables firewall	Drops all network traffic except a short allow-list	Every door locked except the ones on the list
fail2ban	Bans addresses that keep failing to log in	The doorman who turns away repeat offenders
unattended-upgrades	Installs security updates every day by itself	Automatic lock upgrades

Piece	Job in one sentence	Building analogy
Kernel hardening	Safer settings deep in the operating system	Reinforced walls and alarm wiring
auditd	Records changes to accounts, sudo, SSH and firewall files	The logbook
Lynis	Audits the server and gives it a score from 0 to 100	The weekly guard walk with a scorecard
node_exporter + Prometheus	Collect health and security numbers every 15 seconds and raise alerts	Sensors in every building, wired to a control room
Grafana	Draws those numbers as a dashboard	The control-room screens
Molecule	Tests the checklist on three Linux versions before it is used	A test building where every change is rehearsed
GitHub Actions	Runs all tests automatically on every change	The inspector who signs off every new version of the checklist

The flow, in one picture



What you will be able to do after this guide

- Explain each tool and **why it was chosen**, and what the alternatives are.
- Harden a fresh server, measure the improvement, attack it, detect drift and repair it.
- Read and change the Ansible roles, the firewall rules, the alert rules and the CI pipeline.
- Answer the questions in [interview-questions.md](#).

1. Linux server basics

What is it?

A **Linux server** is a computer running Linux with no screen, managed remotely through a terminal. This project targets two of the most common server distributions: **Ubuntu** (24.04 "noble") and **Debian** (12 "bookworm", 13 "trixie"). Ubuntu is built on Debian, so they share the same package manager (`apt`) and the same service manager (`systemd`).

You need five ideas to follow the rest of this guide: **users and groups**, **sudo**, **packages**, **systemd services and timers**, and **logs**.

Why this project needs them

Every hardening step changes one of these five things: - **Users and groups**: who can log in (the `ops` administrator, the `ssh-users` group) - **sudo**: who can become `root` for admin tasks - **Packages**: install security tools (fail2ban, auditd), remove dangerous ones (telnet) - **systemd**: start the protections now and at every boot, and run jobs on a schedule - **Logs**: where SSH failures are written, so fail2ban can read them

Alternatives: Red Hat family systems (RHEL, Rocky, Alma) use `dnf` instead of `apt` and `firewalld` by default. The ideas are the same; the roles here only support Debian and Ubuntu, and say so on the first task.

How it works

Users, groups and root

- Every person and every program runs as a **user**. `root` (user id 0) can do anything.
- A **group** is a named list of users. Permissions can be given to a group instead of to each user.
- `/etc/passwd` lists users, `/etc/group` lists groups, `/etc/shadow` holds password hashes.
- **System users** (like `node_exporter`) have no password and no login shell (`/usr/sbin/nologin`). They exist only so a program runs with few permissions.

sudo

`sudo command` runs one command as root, if `/etc/sudoers` (or a file in `/etc/sudoers.d/`) allows it. A broken `sudoers` file can lock every admin out of root, so it must always be checked with `visudo -c` before it is installed.

Packages

`apt install X` downloads and installs package X and what it needs. `apt purge X` removes it with its config files. `apt update` refreshes the list of available versions.

systemd: services and timers

systemd is the first program started at boot (process 1). It starts everything else. - A **unit** is one thing systemd manages, described in a file such as `/etc/systemd/system/node_exporter.service`. - A **service** is a program systemd starts and restarts if it crashes. `systemctl start/stop/restart/reload/status NAME`. - **enabled** means "start at boot"; **active** means "running now". They are independent: a service can be running but not start at

the next boot. - A **timer** is systemd's scheduler (like cron): `OnCalendar=weekly` starts a matching `.service` on a schedule. - After you change a unit file, systemd needs `daemon-reload` to read it again.

Logs: journald

systemd collects the output of every service in the **journal**. `journalctl -u ssh` shows the SSH server's messages. By default some systems keep the journal only in memory, so it disappears at reboot; this project makes it **persistent**.

Where it is integrated

The `common` role does the basic Linux work. From `roles/common/tasks/main.yml`:

```
- name: Create administrator accounts
  ansible.builtin.user:
    name: "{{ item.name }}"
    shell: /bin/bash
    groups: "{{ common_ssh_group }}"
    append: true
    create_home: true
  loop: "{{ common_admin_users }}"
```

The sudoers file is validated before it replaces anything (`roles/common/templates/sudoers.j2`):

```
- name: Allow administrators to use sudo
  ansible.builtin.template:
    src: sudoers.j2
    dest: /etc/sudoers.d/90-server-baseline
    mode: "0440"
    validate: /usr/sbin/visudo -cf %s
```

Other basics in the same role: removing `telnet`, `rsh-client` and other clear-text tools; the login banner in `/etc/issue.net`; `UMASK 027` in `/etc/login.defs` (new files are not readable by "others"); no core dumps (`/etc/security/limits.d/90-server-baseline.conf`); and persistent logs (`roles/common/templates/journald.conf.j2`):

```
[Journal]
Storage=persistent
Compress=yes
SystemMaxUse=500M
```

Custom systemd units in this project: `server-baseline-firewall.service` (firewall role), `node_exporter.service`, `fail2ban-textfile.timer` and `server-baseline-audit.timer` (Lynis).

Try it

```

make lab-up
docker compose -f lab/compose.yaml exec web-01 bash
id ops
systemctl list-units --type=service --state=running
systemctl list-timers
journalctl -u ssh --since "10 min ago"
exit
# a shell on the lab server web-01
# the admin user and its groups
# what is running
# what runs on a schedule
# SSH server messages

```

After `make harden`, run them again and compare: `ops` is now in `ssh-users`, and there are new services and timers.

Common mistakes

- **Editing** `/etc/sudoers` **without** `visudo`: one typo and nobody can use sudo.
- **"enabled" vs "active"**: starting a service by hand but forgetting to enable it, so it is gone after a reboot. The roles always set both (`state: started` , `enabled: true`).
- **Forgetting** `daemon-reload` after changing a unit file: systemd keeps using the old version.
- **Logs only in memory**: after a reboot the evidence of an attack is gone.

Check yourself

1. What is the difference between an *enabled* and an *active* service?
2. Why does `node_exporter` run as its own user instead of root?
3. What does `validate: /usr/sbin/visudo -cf %s` protect against?

▼ Answers

1. **Enabled** means systemd starts it at boot; **active** means it is running right now. You usually want both.
2. **Least privilege**: if the program had a bug that let an attacker control it, the attacker would only get the rights of a user with no shell and no password, not root.
3. Ansible writes the new sudoers file to a temporary path, runs `visudo -c` on it, and only replaces the real file if the check passes. A broken file never reaches the server, so sudo keeps working.

2. SSH and keys

What is it?

SSH (Secure Shell) is how you log in to a remote Linux server and run commands over an encrypted connection. The program on the server is **sshd** (the SSH daemon); the program you type into is `ssh`.

SSH supports several ways to prove who you are. The two that matter: - **Password**: you type a secret the server also knows (as a hash). - **Public key**: you hold a **private key** file; the server holds the matching **public key** in `~/.ssh/authorized_keys`.

Why this project uses it (and hardens it)

SSH is the front door. It is also how **Ansible** reaches every server, so the hardening must make the door strong **without locking Ansible out**.

Default on Debian/Ubuntu	After the baseline	Why
Password login allowed	Keys only (<code>AuthenticationMethods publickey</code>)	Passwords can be guessed; a 256-bit key cannot
<code>root</code> may log in with a key	No root login at all	Admins log in as themselves and use sudo, so every action has a name
Every user may log in	Only the <code>ssh-users</code> group	A service account with a weak setting cannot become a way in
6 tries per connection	3 tries , 30 s to log in	Slows down guessing
Tunnels and forwarding allowed	Off	A login is a shell, not a path into the internal network
Older algorithms allowed	Modern ciphers, key exchanges and MACs only	No CBC ciphers, no SHA-1 MACs

The lab's end-to-end test proves this from the outside: before hardening the server offers `publickey,password`; after, it offers only `publickey`, a correct password is rejected, and even a **valid key copied into root's account is rejected**.

Alternatives: a VPN or bastion host in front of SSH, short-lived SSH certificates (for example from HashiCorp Vault or Teleport), or no SSH at all with a cloud session manager (AWS SSM). Those add infrastructure; key-only SSH with fail2ban is the baseline every server needs anyway.

How key login works

```
your laptop / controller          server
private key (never leaves)       ~/.ssh/authorized_keys: public key
| 1. "I want to log in as ops with this public key" →
| ← 2. "prove it: sign this random challenge"
| 3. signature made with the PRIVATE key →
|                                4. checks the signature with the PUBLIC key → OK
```

The private key is never sent. A stolen `authorized_keys` file is useless to an attacker.

sshd_config, drop-ins and "first value wins"

sshd reads `/etc/ssh/sshd_config`. Modern versions start with:

```
Include /etc/ssh/sshd_config.d/*.conf
```

Files in that folder are read **in alphabetical order**, and for most settings **the first value sshd reads wins**. That is why the baseline's file is called `00-server-baseline.conf`: it is read before anything else, including `50-cloud-init.conf` that cloud images ship with `PasswordAuthentication yes`.

`sshd -t` and `sshd -T`

- `sshd -t` **tests** a configuration and fails on any error. A broken config plus a restart can mean **nobody can log in again**.
- `sshd -T` prints the **effective** settings after reading every file. This is the only reliable answer to "is password login really off?"

Ubuntu 24.04: socket activation

On Ubuntu 24.04, sshd is started on demand by `ssh.socket`. Debian runs `ssh.service` all the time. The baseline switches Ubuntu to the classic always-running `ssh.service`, so both distributions behave the same and the port lives in one place.

Where it is integrated

`roles/ssh_hardening/templates/sshd-baseline.conf.j2` (excerpt):

```
AllowGroups {{ ssh_hardening_allow_groups | join(' ') }}
PermitRootLogin no
PubkeyAuthentication yes
AuthenticationMethods publickey
PasswordAuthentication no
KbdInteractiveAuthentication no
MaxAuthTries {{ ssh_hardening_max_auth_tries }}
AllowTcpForwarding no
KexAlgorithms {{ ssh_hardening_kex_algorithms | join(',') }}
```

`roles/ssh_hardening/tasks/main.yml` installs it **only if sshd accepts it**, and reloads sshd through a handler:

```
- name: Install the hardened SSH settings
  ansible.builtin.template:
    src: sshd-baseline.conf.j2
    dest: /etc/ssh/sshd_config.d/00-server-baseline.conf
    mode: "0600"
    validate: /usr/sbin/sshd -t -f %s
  notify: Reload sshd
```

Two **lockout guards** run before that: - The `common` role refuses to continue if `common_admin_users` is empty or a user has no key. - The `ssh_hardening` role refuses to restrict SSH to groups that have no members.

The role also removes weak Diffie-Hellman groups (smaller than 3071 bits) from `/etc/ssh/moduli`.

The verification (`roles/ssh_hardening/tasks/verify.yml`) reads `sshd -T` and asserts each setting, for example:

```
- ssh_hardening_settings.passwordauthentication == 'no'
- ssh_hardening_settings.permitrootlogin == 'no'
- "'cbc' not in ssh_hardening_settings.ciphers"
```

Try it

```
# Which login methods does web-01 offer to a stranger? (from the attacker container)
docker compose -f lab/compose.yaml exec attacker \
  ssh -o BatchMode=yes -o StrictHostKeyChecking=no -o PreferredAuthentications=none probe@web-01
# before hardening: Permission denied (publickey,password).  after: Permission denied (publickey).

# The effective settings, straight from sshd
docker compose -f lab/compose.yaml exec web-01 sh -c "sshd -T | grep -E
'passwordauth|permitrootlogin|allowgroups|maxauthtries'"

# Ansible's own key login (from the controller)
docker compose -f lab/compose.yaml exec controller ssh -i /keys/id_ed25519 -o
StrictHostKeyChecking=accept-new -o UserKnownHostsFile=/keys/known_hosts ops@web-01 hostname
```

Common mistakes

- **Restarting sshd with an untested config.** Always `sshd -t` first; the role does it with `validate: .`
- **Editing `sshd_config` and wondering why nothing changed:** an earlier drop-in already set the value. Check with `sshd -T`.
- **Disabling password login before a key works.** The lockout guards exist for exactly this.
- **Closing your only session** after changing SSH. Keep one session open and test with a second one.
- **Copying the private key to servers.** Only the public key goes onto servers.

Check yourself

1. Why is the baseline file named `00-server-baseline.conf` and not `99-... ?`
2. What is the difference between `sshd -t` and `sshd -T`?
3. An attacker steals a server's `authorized_keys`. Can they log in?
4. Why is root login disabled even with keys?

▼ Answers

1. For most settings sshd uses **the first value it reads**. `00-` is read first, so other drop-ins (like `50-cloud-init.conf`) cannot override it.
2. `-t` **tests** the configuration for errors; `-T` prints the **effective** settings after all files are combined.
3. **No**. It contains only public keys. Logging in needs the private key, which never leaves the admin's machine.
4. So every action is done by a named person through sudo (and logged), and because `root` is the one account name every attacker knows.

3. Ansible

What is it?

Ansible is a tool that configures computers from a description written in YAML. You describe the **desired state** ("package fail2ban is installed", "this file has this content", "this service is running"), and Ansible connects to each server over **SSH**, compares, and changes only what differs.

It is **agentless**: nothing needs to be installed on the servers except SSH and Python.

Why this project uses it

Hardening is a long list of small settings that must be **identical on every server** and **stay that way**. Ansible gives: - **Repeatability**: the same playbook produces the same server every time. - **Idempotence**: running it again changes nothing if nothing drifted. That makes the playbook double as a **drift check**. - **Review**: the security baseline is code in Git, reviewed in pull requests and tested in CI.

Alternatives: - **Shell scripts**: quick, but not idempotent without a lot of care, and hard to test. - **Chef / Puppet / Salt**: need an agent or server on each machine. - **Golden images (Packer)**: good for new servers, but don't fix drift on running ones. Many teams combine Packer *and* Ansible. - **CIS benchmark roles (e.g. ansible-lockdown)**: very complete, but hundreds of controls. This project builds a smaller baseline you can read in an afternoon, and measures it with Lynis.

How it works: the vocabulary

Term	Meaning	In this repo
Control node	The machine that runs Ansible	The <code>controller</code> container (lab/controller/Dockerfile)
Inventory	The list of servers, in groups, with variables	inventories/lab/hosts.yml : group <code>baseline</code> with web-01 and db-01
Playbook	A YAML file with one or more plays : "on these hosts, do this"	playbooks/site.yml , verify.yml , audit.yml
Task	One step, calling one module (<code>apt</code> , <code>template</code> , <code>user</code> , <code>systemd_service</code> , ...)	Every file in <code>roles/*/tasks/</code>
Role	A folder of tasks, defaults, templates, handlers for one job	9 roles in roles/
Variables / defaults	Values tasks use. defaults/main.yml = lowest priority, easy to override	roles/*/defaults/main.yml , overridden in inventories/lab/group_vars/baseline.yml
Facts	Information Ansible collects about each server first (OS, CPU, virtualization)	<code>ansible_facts['distribution']</code> , <code>ansible_facts['virtualization_type']</code>
Template	A file with <code>{{ variables }}</code> and <code>{% loops %}</code> (Jinja2)	roles/*/templates/*.j2

Term	Meaning	In this repo
Handler	A task that runs only if notified by a task that changed something, once, at the end	Reload sshd , Reload firewall , Restart fail2ban
Tags	Labels to run only part of a playbook	--tags ssh , --tags firewall

The playbook of this project

From `playbooks/site.yml`:

```
- name: Apply the server baseline
  hosts: baseline
  become: true
  # One server at a time: a mistake stops after the first server instead of all of them.
  serial: "{{ baseline_serial | default(1) }}"
  max_fail_percentage: 0

  roles:
    - role: common
      tags: [common]
    - role: kernel_hardening
      tags: [kernel]
    - role: ssh_hardening
      tags: [ssh]
  ...
```

- `become: true` = use sudo on the server.
- `serial: 1` = a **rolling** change: one server at a time. With `max_fail_percentage: 0`, the first failure stops the whole run, so a bad change breaks one server, not the fleet.
- At the end, `post_tasks` run `meta: reset_connection` and `wait_for_connection`: a **brand-new SSH connection** proves that SSH and the firewall still let the admin in.

Idempotence: "changed" must mean something

Ansible reports every task as `ok` (already right), `changed` (fixed now), `skipped` or `failed`. The rule in this project: **a second run must report 0 changes**. The Molecule test, the lab end-to-end test and the CI VM test all check it.

Modules like `template` and `apt` are idempotent on their own. Raw commands are not, so the project writes them carefully. From `roles/kernel_hardening/tasks/main.yml`, read first, then write only what differs:

```
- name: Read the running kernel values
  ansible.builtin.command: sysctl -n {{ item.key }}
  register: kernel_hardening_current
  changed_when: false      # reading never "changes" anything
  check_mode: false        # also read during --check

- name: Apply kernel values that differ
  ansible.builtin.command: sysctl -w {{ item.item.key }}={{ item.item.value }}
  when: item.stdout | trim != item.item.value | string
```

Check mode and diff mode: the drift check

`ansible-playbook --check --diff` **pretends**: modules report what they *would* change and show a diff, but change nothing. `scripts/drift-check.sh` uses this: exit 0 means every server matches the baseline; exit 2 means drift, with the diff showing what and where. It lists the *tasks* that would change (`server | task`) and does not count the handlers they would trigger: one drifted SSH setting also "changes" the `Reload sshd` handler, but that is a consequence, not a second drift.

`validate:` : never install a broken file

Several templates have a `validate:` command. Ansible writes the new file to a temporary path, runs the command with `%s` replaced by that path, and **only installs the file if the command succeeds**:

File	Validated with
<code>/etc/sudoers.d/90-server-baseline</code>	<code>visudo -cf %s</code>
<code>/etc/ssh/sshd_config.d/00-server-baseline.conf</code>	<code>sshd -t -f %s</code>
<code>/etc/nftables.d/server-baseline.nft</code>	<code>nft -c -f %s</code>

Verification as code

Each role also has `tasks/verify.yml`, and `playbooks/verify.yml` runs them all with `include_role ... tasks_from: verify.yml`. The same checks run in Molecule, in the lab test and on the CI VM.

Why Ansible runs in a container here

The `controller` image pins `ansible-core 2.21.4`, `ansible-lint`, `molecule`, the collections (`ansible.posix`, `community.general`, `community.docker`) and the Docker CLI (`requirements.txt`, `requirements.yml`). Everyone, and CI, runs the same versions. Ansible's control node also does not run natively on Windows.

Try it

```
make lab-up
docker compose -f lab/compose.yaml exec controller ansible baseline -m ansible.builtin.ping
docker compose -f lab/compose.yaml exec controller ansible web-01 -m ansible.builtin.setup -a
'filter=ansible_distribution*'
make harden                                # first run: many "changed"
make harden                                # second run: changed=0 on both servers
scripts/harden.sh --limit web-01 --tags ssh # only one role, one server
make drift                                  # check mode: what differs? (nothing)
```

Common mistakes

- **Using `command` / `shell` for everything:** it always reports "changed" and breaks idempotence. Use a module, or set `changed_when` honestly.
- **Restarting services in every run:** use handlers, which run only when something changed.
- **Putting secrets in variables in Git:** this project generates the SSH key at runtime in a Docker volume, never in the repo.
- **Inventory variables for play keywords:** `serial` is evaluated before hosts exist, so it cannot come from `group_vars`; Molecule passes it as an extra var.
- **Testing only the first run.** The second run is where bugs in idempotence show up.

Check yourself

1. What is the difference between a task and a handler?
2. Why does `site.yml` use `serial: 1` with `max_fail_percentage: 0` ?
3. What does `--check --diff` do, and how does this project use it?
4. Why do the "read" tasks have `check_mode: false` ?

▼ Answers

1. A task always runs. A handler runs **only if a task notified it and that task changed something**, and only once, after the tasks (or at `flush_handlers`).
2. To roll the change out one server at a time and stop at the first failure, so a mistake affects one server instead of all of them.
3. It runs the playbook without changing anything and shows what would change. `scripts/drift-check.sh` uses it to detect drift: 0 changes means "matches the baseline".
4. In check mode, `command` tasks are normally skipped. A read-only command must still run, otherwise the next tasks have no data to compare and the drift check would miss differences.

4. Firewall with nftables

What is it?

A **firewall** decides which network packets may reach the server. **nftables** is the modern Linux firewall built into the kernel; its command is `nft`. It replaces the older `iptables`.

Why this project uses it

Without a firewall, **every port any program opens is reachable from the internet**. The baseline turns this around: **drop everything, then allow a short list**. In the lab: - port **22 (SSH)** from anywhere (fail2ban guards it, chapter 5) - port **9100 (node_exporter metrics)** only from Prometheus at `172.30.0.10`

The lab test proves it: Prometheus can scrape port 9100, the attacker container cannot.

Alternatives: `ufw` (Ubuntu's simple front end), `firewalld` (Red Hat), cloud security groups. Cloud security groups are a good extra layer, but they don't exist on bare metal and don't protect against traffic from inside the same network. Raw nftables works the same on Debian and Ubuntu and teaches what the front ends hide.

How it works

```
table inet server_baseline           ← a named container of rules; "inet" = IPv4 + IPv6
chain input                          ← attached to the "input" hook: packets for this machine
  policy drop                        ← anything no rule accepts is dropped
  rule: ct state established,related accept ← replies to connections we started
  rule: iif "lo" accept               ← the machine talking to itself
  rule: ip saddr { 0.0.0.0/0 } tcp dport 22 accept
  rule: ip saddr { 172.30.0.10/32 } tcp dport 9100 accept
  (everything else → policy drop, logged at most 10 times a minute)
```

- **Table:** a namespace for chains. Several tables can exist side by side (fail2ban creates its own, Docker creates its own).
- **Chain with a hook:** the kernel sends packets to it at that point (input, forward, output).
- **Policy:** what happens at the end of the chain.
- **Connection tracking (`ct state`):** the kernel remembers connections, so answers to outgoing requests (updates, DNS) come back without extra rules.

Why the baseline uses its own table and no `flush ruleset`

Many examples start with `flush ruleset`, which **deletes every rule on the machine**, including Docker's and fail2ban's. The baseline only replaces **its own table**:

```
table inet server_baseline
delete table inet server_baseline
table inet server_baseline { ... }
```

The first line creates the table if it is missing, so the delete never fails; the last line rebuilds it. All three happen in one atomic transaction.

`nft -c` : check before applying

`nft -c -f file` parses the file **and checks it against the running kernel** without changing anything. A typo (or an impossible address like `300.1.1.1`) is rejected before it can cut off SSH.

Where it is integrated

`roles/firewall/templates/server-baseline.nft.j2` builds the rules from the `firewall_rules` variable:

```
{% for rule in firewall_rules %}
  # {{ rule.name }}
{% if rule.sources_v4 | default([]) | length > 0 %}
  ip saddr { {{ rule.sources_v4 | join(', ') }} } tcp dport {{ rule.port }} accept comment "{{
rule.name }}"
{% endif %}
```

The lab rules live in `inventories/lab/group_vars/baseline.yml`:

```
firewall_rules:
- name: ssh
  port: 22
  sources_v4: ["0.0.0.0/0"]
  sources_v6: [":::/0"]
- name: node-exporter
  port: 9100
  sources_v4: ["{{ lab_prometheus_ip }}/32"]
  sources_v6: []
```

`roles/firewall/tasks/main.yml` has three safety features: 1. **Lockout guard**: fails if no rule opens the SSH port (`firewall_ssh_port`). 2. **validate**: `/usr/sbin/nft -c -f %s` on the template. 3. **A runtime check**: a systemd oneshot service stays "active" even if someone deletes the rules by hand, so the role asks the kernel directly (`nft list table inet server_baseline`) and reloads if the table is missing. This is how the drift test catches a deleted firewall.

The rules are loaded at boot by its own unit, `server-baseline-firewall.service.j2`: `ExecStart` / `ExecReload` load the file, `ExecStop` deletes only this table.

`firewall_manage_forward` adds a `forward` chain with policy drop (a server is not a router). It is set to `false` on the CI VM, which also runs Docker containers that need forwarding.

Try it

```
docker compose -f lab/compose.yaml exec web-01 nft list table inet server_baseline
docker compose -f lab/compose.yaml exec web-01 nft list tables                # yours, fail2ban's, others
# From the attacker: metrics port blocked (times out)
docker compose -f lab/compose.yaml exec attacker curl -m 4 http://web-01:9100/metrics
# From Prometheus' point of view: Status → Targets at http://localhost:9090 shows both servers UP
# Dropped packets are logged by the kernel:
docker compose -f lab/compose.yaml exec web-01 sh -c "journalctl -k | grep nft-baseline-drop | tail -3"
```

Common mistakes

- `flush ruleset` in a firewall file on a Docker host: containers lose networking and DNS.
- **Forgetting** `ct state established,related accept`: outgoing connections never get their answers.

- **Applying a firewall over SSH without a rule for SSH.** The lockout guard blocks this.
- **Trusting** `systemctl status` : a oneshot unit says "active" even if the rules were deleted by hand. Check the kernel.

Check yourself

1. Why does the template start with `table inet server_baseline` followed by `delete table inet server_baseline` ?
2. Port 9100 is open only to `172.30.0.10/32` . What is `/32` ?
3. What happens to a packet for port 8080?
4. Why is the forward chain switched off on the CI VM?

▼ Answers

1. The first line creates the table if it doesn't exist, so the delete always succeeds; then the table is rebuilt. Only this table is replaced; other tables (fail2ban, Docker) are untouched.
2. A network of exactly one address: only Prometheus.
3. No rule accepts it, so the chain's `policy drop` drops it (and it may be logged as `nft-baseline-drop:`).
4. The runner also runs Docker containers whose traffic is forwarded through the host. A forward chain with `policy drop` would cut them off.

5. fail2ban

What is it?

fail2ban reads log messages, counts failures per IP address, and **bans** addresses that fail too often by adding a firewall rule. After the ban time, it removes the rule again.

Why this project uses it

Port 22 must stay open for administrators, so bots will knock on it all day. Key-only SSH means they cannot get in, but they still fill logs and use CPU. fail2ban cuts them off after a few attempts, and its counters make attacks **visible** on the dashboard.

Alternatives: **CrowdSec** (shares attacker lists between users), **sshguard** (smaller, similar idea), SSH on a non-standard port (reduces noise, but is not security), or keeping SSH off the internet entirely (VPN, bastion). fail2ban is the most widely known and is packaged on every distribution.

How it works

A **jail** = a log source + a filter (which messages count as a failure) + an action (how to ban).

Setting	Baseline value	Meaning
<code>maxretry</code>	5	failures before a ban
<code>findtime</code>	10m	...within this window
<code>bantime</code>	1h	first ban length
<code>bantime.increment</code>	true	repeat offenders: 1h, 2h, 4h ... up to <code>bantime.maxtime</code> (1w)
<code>ignoreip</code>	127.0.0.1/8, ::1, the controller	never banned
<code>backend</code>	systemd	read sshd's messages from the journal (no <code>/var/log/auth.log</code> needed)
<code>mode</code>	aggressive	also count scanners that disconnect before even trying a key
<code>banaction</code>	<code>nftables[type=multiport]</code>	ban by adding the address to fail2ban's own nftables table

```
sshd: "Invalid user guess3 from 172.30.0.66 port 51234" → systemd journal
fail2ban sshd jail reads the journal, filter matches, counts 172.30.0.66: 1, 2, 3, 4, 5
→ ban: adds 172.30.0.66 to a set in table inet f2b-table (nftables)
→ further packets from 172.30.0.66 to port 22 are rejected
```

Two things the lab showed that surprise people:

- **One connection can count twice.** In `aggressive` mode a single failed connection usually matches two log lines ("Invalid user ..." and "Connection closed by invalid user ..."), so 5 failures can be reached after only 3 connections.
- **Longer bans are for bans that run out on their own.** `bantime.increment` makes each new ban of a known offender longer (1 h, 2 h, 4 h ...). In the lab, an address that was unbanned *by hand* and then

banned again got the normal 1 h ban (`fail2ban-client get sshd banip --with-time` shows the length), so a manual unban effectively gives the address a fresh start. Fix a colleague's key *before* unbanning them, or they will simply be banned again.

Why the controller is on the ignore list

If a test (or a typo) made the Ansible controller fail five times, fail2ban would ban it and **Ansible would be locked out of every server**. `fail2ban_ignoreip_extra` puts the controller's address on the never-ban list.

Where it is integrated

`roles/fail2ban/templates/jail.local.j2`:

```
[DEFAULT]
ignoreip = {{ (fail2ban_ignoreip + fail2ban_ignoreip_extra) | join(' ') }}
bantime = {{ fail2ban_bantime }}
findtime = {{ fail2ban_findtime }}
maxretry = {{ fail2ban_maxretry }}
banaction = nftables[type=multiport]

[sshd]
enabled = true
backend = systemd
mode = aggressive
```

It is installed as `/etc/fail2ban/jail.d/server-baseline.local` (`.local` files override the package's `.conf` files and survive package upgrades). The package `python3-systemd` lets fail2ban read the journal.

Metrics: a small script, `fail2ban-textfile.sh.j2`, runs every minute from a systemd timer and writes `fail2ban_up`, `fail2ban_banned_current`, `fail2ban_banned_total` and `fail2ban_failed_current` to `/var/lib/node_exporter/textfile/fail2ban.prom`. `node_exporter` publishes them (chapter 10), and the `SSHAttackerBanned` alert fires when someone is banned.

Try it

```
# Attack db-01 from the attacker container: 8 logins with unknown user names
for i in 1 2 3 4 5 6 7 8; do
    docker compose -f lab/compose.yaml exec -T attacker ssh -o BatchMode=yes -o StrictHostKeyChecking=no -o ConnectTimeout=5 guess$i@db-01 true
done
docker compose -f lab/compose.yaml exec db-01 fail2ban-client status sshd      # Banned IP list:
172.30.0.66
docker compose -f lab/compose.yaml exec db-01 nft list table inet f2b-table  # the ban, as a firewall rule
docker compose -f lab/compose.yaml exec db-01 cat /var/lib/node_exporter/textfile/fail2ban.prom
# Unban
docker compose -f lab/compose.yaml exec db-01 fail2ban-client set sshd unbanip 172.30.0.66
```

Common mistakes

- **Banning yourself**, for example from the office or the automation server. Use `ignoreip`.

- **Wrong log source:** on modern Debian there is no `/var/log/auth.log` by default, so a file-based jail never sees anything. Use `backend = systemd`.
- **Editing `jail.conf`:** a package upgrade overwrites it. Put changes in `jail.d/*.local`.
- **Treating fail2ban as the defence.** It reduces noise; key-only SSH is the defence.

Check yourself

1. With the baseline values, what happens on the 6th failed login within 10 minutes?
2. Why is the Ansible controller in `ignoreip`?
3. How does a ban reach the Grafana dashboard?

▼ Answers

1. The address was already banned after the 5th failure, so the 6th connection is rejected by the nftables rule fail2ban added.
2. If it were banned, Ansible could no longer reach the servers, and nobody could repair them with automation.
3. sshd logs the failures to the journal → fail2ban bans → the textfile script writes `fail2ban_banned_current` → node_exporter publishes it → Prometheus scrapes it (and fires `SSHAttackerBanned`) → Grafana shows it in "Addresses banned now".

6. Automatic security updates

What is it?

unattended-upgrades is the Debian/Ubuntu tool that installs updates automatically. Two **systemd timers** trigger it every day: - `apt-daily.timer` → refreshes the package lists (like `apt update`) - `apt-daily-upgrade.timer` → installs the allowed updates

Why this project uses it

Most break-ins use **known** vulnerabilities for which a fix already exists. A server that is not updated stays vulnerable for weeks. Automatic **security** updates close that window within a day, without a person remembering to log in.

Alternatives: patching with Ansible on a schedule (more control, needs someone to run it), a patch-management service (AWS Systems Manager Patch Manager, Landscape), or immutable servers that are replaced with a new image instead of patched. Automatic security updates are the baseline that works everywhere.

How it works

The settings live in small files in `/etc/apt/apt.conf.d/`, read in alphabetical order. **Later files override earlier ones.**

File	Owner	Content
<code>20auto-upgrades</code>	this baseline	turns the daily refresh and install on
<code>50unattended-upgrades</code>	the distribution	which updates: on Debian and Ubuntu, security updates only
<code>52server-baseline</code>	this baseline	how : reboot or not, cleanup, logging

The baseline deliberately **keeps the distribution's origin list** (`50unattended-upgrades`) instead of writing its own. The distribution maintains it, so it stays correct across releases.

Reboots: some updates (kernel, C library) only take effect after a reboot. `auto_updates_reboot` is **false** by default: a reboot should be a decision. `needrestart` (installed by the `common` role) restarts services that use updated libraries.

Where it is integrated

`roles/auto_updates/templates/20auto-upgrades.j2` :

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
APT::Periodic::AutocleanInterval "{{ auto_updates_autoclean_interval }}";
```

`roles/auto_updates/templates/52server-baseline.j2` :

```
Unattended-Upgrade::Automatic-Reboot "{{{ 'true' if auto_updates_reboot | bool else 'false' }}}";
Unattended-Upgrade::Remove-Unused-Dependencies "...";
Unattended-Upgrade::SyslogEnable "true";
```

The verification ([roles/auto_updates/tasks/verify.yml](#)) does not trust the files; it reads the **final merged configuration** with `apt-config dump` and checks that both timers are enabled. The `SecurityServiceNotRunning` alert also fires if `apt-daily-upgrade.timer` stops.

Try it

```
docker compose -f lab/compose.yaml exec web-01 sh -c "apt-config dump | grep -E 'Periodic|Automatic-Reboot'"
docker compose -f lab/compose.yaml exec web-01 systemctl list-timers 'apt-daily*'
# What would be installed right now (changes nothing):
docker compose -f lab/compose.yaml exec web-01 unattended-upgrade --dry-run --debug 2>&1 | tail -20
```

Common mistakes

- **Enabling all updates, not only security ones**, on production servers: a feature update can change behaviour overnight.
- **Turning on automatic reboots** without knowing what the server does. Plan a maintenance window first.
- **Disabling the timers** "because apt was locked once". Then nothing updates any more; the alert exists for this.

Check yourself

1. Which file decides *which* updates are installed, and why doesn't the baseline replace it?
2. Why is `Automatic-Reboot` false by default?
3. How would you notice that automatic updates stopped working?

▼ Answers

1. The distribution's `50unattended-upgrades`. The distribution keeps it correct for each release; the baseline only adds behaviour settings in a later file.
2. A reboot interrupts the service; it should be planned. `needrestart` covers most cases without a reboot.
3. The `SecurityServiceNotRunning` alert fires if `apt-daily-upgrade.timer` is not active, and the "Security services" table in Grafana shows it red. Lynis also warns about missing updates.

7. Kernel hardening

What is it?

The **kernel** is the core of Linux. It has hundreds of settings, called **sysctl** parameters, visible as files under `/proc/sys/`. For example `net.ipv4.conf.all.accept_redirects` is `/proc/sys/net/ipv4/conf/all/accept_redirects`.

`sysctl -n KEY` reads a value, `sysctl -w KEY=VALUE` changes it **until the next reboot**, and files in `/etc/sysctl.d/` apply values **at every boot**.

The kernel can also load **modules** (drivers for filesystems, network protocols). Files in `/etc/modprobe.d/` can block modules from loading.

Why this project uses it

Several defaults are convenient but help attackers:

Setting	Baseline	Protects against
<code>net.ipv4.conf.*.accept_redirects</code>	0	Fake "use this router instead" messages that redirect traffic
<code>net.ipv4.conf.*.rp_filter</code>	1	Packets with spoofed source addresses
<code>net.ipv4.tcp_syncookies</code>	1	SYN-flood attacks
<code>net.ipv4.conf.*.log_martians</code>	1	Logs impossible source addresses
<code>kernel.kptr_restrict</code>	2	Hides kernel memory addresses that make exploits easier
<code>kernel.dmesg_restrict</code>	1	Only root can read the kernel log
<code>kernel.yama.ptrace_scope</code>	1	A program cannot inspect another program's memory (password theft)
<code>kernel.unprivileged_bpf_disabled</code>	1	Removes a common local-exploit path
<code>fs.suid_dumpable</code>	0	Privileged programs never write memory dumps
<code>fs.protected_symlinks</code> / <code>_hardlinks</code>	1	Link tricks in shared folders like <code>/tmp</code>

Blocked modules: rare filesystems (`cramfs` , `hfs` , `jffs2` , ...) and rare network protocols (`dccp` , `sctp` , `rds` , `tipc`) that a normal server never uses but that have had kernel bugs.

Not set on purpose: `net.ipv4.ip_forward`. Docker and Kubernetes hosts need forwarding, so it is left to the machine's role.

Alternatives: CIS benchmark roles set many more values; a hardened image bakes them in. This list follows the common recommendations Lynis checks for (test `KRNL-6000`).

How it works: and why containers are different

A container **shares the host's kernel**. Most kernel settings (`kernel.*` , `fs.*`) are **global**: changing them inside a privileged container would change them for the **host and every other container**. Only some network

settings (`net.ipv4.*`, `net.ipv6.*`) exist **per container** (each container has its own network namespace).

So the role decides at run time:

```
# roles/kernel_hardening/defaults/main.yml
kernel_hardening_in_container: >-
  {{ ansible_facts['virtualization_role'] == 'guest'
    and ansible_facts['virtualization_type'] in ['docker', 'podman', 'container', 'containerd', 'lxc']
  }}
kernel_hardening_container_safe_prefixes:
  - net.ipv4.
  - net.ipv6.
```

- **In a container** (Molecule, the lab): the persistent file is written with **all** settings, but only `net.*` settings are applied live.
- **On a real machine or VM** (the CI `vm` job): **every** setting is applied and verified live.

The verify step says which case it checked, for example "... kernel settings checked live (container: per-container network settings only)".

Where it is integrated

- `roles/kernel_hardening/defaults/main.yml`: the list of settings. `optional: true` marks settings some kernels don't have.
- `roles/kernel_hardening/templates/sysctl.conf.j2` writes `/etc/sysctl.d/90-server-baseline.conf`. Optional keys get a leading `-`, which tells `sysctl` "skip this if the kernel doesn't have it":

```
{{ '-' if s.optional | default(false) else '' }}{{ s.key }} = {{ s.value }}
```

- `roles/kernel_hardening/templates/modprobe.conf.j2`: `install dccp /bin/false` plus `blacklist dccp`. `blacklist` alone only stops automatic loading; `install ... /bin/false` also stops explicit loading.
- `roles/kernel_hardening/tasks/main.yml`: reads each live value and writes only the ones that differ (idempotent), plus a check-mode task that **reports** differences for the drift check.

Try it

```
docker compose -f lab/compose.yaml exec web-01 cat /etc/sysctl.d/90-server-baseline.conf
docker compose -f lab/compose.yaml exec web-01 sysctl net.ipv4.conf.all.accept_redirects
# Make the server drift, then let the drift check find it:
docker compose -f lab/compose.yaml exec web-01 sysctl -w net.ipv4.conf.all.accept_redirects=1
make drift          # reports the difference, changes nothing
make harden         # puts it back
```

Common mistakes

- **Changing global kernel settings inside a privileged container:** you silently change the host.
- **Using only `sysctl -w`:** gone after a reboot. Use `/etc/sysctl.d/`.
- **Disabling forwarding on a container host:** Docker networking stops.

- **Blocking udf** : some clouds (for example Azure) deliver provisioning data on a UDF disk, so it is not in the list.

Check yourself

1. Why does the role apply only `net.*` settings inside containers?
2. What is the difference between `blacklist sctp` and `install sctp /bin/false`?
3. What does the leading `-` mean in `-kernel.yama.ptrace_scope = 1`?

▼ Answers

1. Containers share the host kernel. Only network settings are per container; changing the others would change the host and all its containers.
2. `blacklist` stops automatic loading only; `install ... /bin/false` makes every attempt to load it run `/bin/false` instead, so it never loads.
3. "Ignore errors for this key": if the kernel doesn't have the setting, boot continues without an error.

8. auditd

What is it?

The Linux kernel has an **audit system** that can record events: "file X was written", "program Y was run as root". **auditd** is the service that receives these records and writes them to `/var/log/audit/audit.log`. You search them with `ausearch` and summarise them with `aureport`.

Why this project uses it

After an incident, the first questions are: *who changed this account? who edited the SSH settings? who ran what as root?* Normal logs often don't say. auditd records it **at the kernel level**, so it works even if a program writes no log at all. Security standards (CIS, ISO 27001, PCI DSS) ask for it.

Alternatives: **Falco** or **Tetragon** (eBPF-based, popular on Kubernetes), osquery, commercial EDR agents. auditd is built into every Linux distribution and is what Lynis checks for (test `ACCT-9628`).

How it works

Two kinds of rules, in `/etc/audit/rules.d/*.rules`, combined and loaded by `augenrules --load`:

```
-w /etc/sudoers -p wa -k privilege
  |               |               | key: a tag to search by
  |               | watch for writes (w) and attribute changes (a)
  |_____ watch this file or folder

-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k root-commands
  |_____ on every program start (execve) running as root (euid=0) by a logged-in person (auid >= 1000)
```

`auid` is the **login user id**: it stays the same after `sudo`, so the record shows *which person* became root.

Immutable mode (`-e 2`) locks the rules until the next reboot, so an attacker with root cannot switch auditing off quietly. It also means every rule change needs a reboot, so `auditd_immutable` is `false` by default.

Why auditd cannot run in the lab containers

The kernel audit system is **not namespaced**: there is one per kernel, owned by the host. Inside a container, `auditctl` fails with `Operation not permitted`. So: - In containers, the role installs the rules but leaves the service disabled, and says so. - The CI **vm job** runs the baseline on a real GitHub runner VM, where auditd runs, the rules are loaded, and the test writes to `/etc/hosts` and checks that `ausearch -k network` finds the event. - In Prometheus, the `AuditdNotRunning` alert only fires on servers with `server_baseline_info{kind="machine"}`.

Where it is integrated

`roles/auditd/defaults/main.yml` lists the watched files:

```
auditd_manage_service: "{{ not (auditd_in_container | bool) }}"
auditd_watches:
  - {path: /etc/passwd, key: identity}
  - {path: /etc/shadow, key: identity}
  - {path: /etc/sudoers.d/, key: privilege}
  - {path: /etc/ssh/ssh_config.d/, key: sshd}
  - {path: /etc/nftables.d/, key: firewall}
  ...
```

`roles/auditd/templates/server-baseline.rules.j2` turns them into rules and adds time changes, hostname changes, kernel module loading and root commands.

`roles/auditd/handlers/main.yml` has a detail worth knowing: auditd refuses `systemctl restart` (its unit has `RefuseManualStop=yes`), so the handler uses `service auditd restart`, with a `# noqa` comment explaining it to ansible-lint.

The VM test: `scripts/vm-test.sh`

```
echo "# audit test $(date +%s)" | sudo tee -a /etc/hosts >/dev/null
sudo ausearch -k network -f /etc/hosts -ts recent
```

Try it

In the lab you can look at the rules, but not load them:

```
docker compose -f lab/compose.yaml exec web-01 cat /etc/audit/rules.d/50-server-baseline.rules
```

On a real Linux VM with the baseline applied:

```
sudo auditctl -l                # rules loaded in the kernel
sudo usermod -c "test" ops      # change an account
sudo ausearch -k identity -i | tail # who did it, when, with which program
sudo aureport --summary
```

Common mistakes

- **Watching too much** (for example every `execve` of every user): the log grows very fast and the disk fills up. The baseline rotates logs (`max_log_file_action = ROTATE`) and alerts on full disks.
- **Watching paths that don't exist** (for example `/var/log/lastlog` on Debian 13): rule loading fails.
- **Expecting auditd in containers.** Test it on a VM.
- **Immutable mode during development:** every change needs a reboot.

Check yourself

1. What does `-p wa` mean in a watch rule?
2. Why is `audit` more useful than the normal user id after `sudo`?
3. How does this project prove auditd works, given the lab can't run it?

▼ Answers

1. Record **w**rites and **a**tttribute changes (permissions, owner) to the watched path.
2. After `sudo` , the user id is 0 (root), but `audit` still holds the id of the person who logged in, so the record says who did it.
3. The CI `vm` job applies the baseline to the GitHub runner VM, checks every watch rule is loaded (`auditctl -l`), changes `/etc/hosts` , and checks `ausearch` finds the event.

9. Lynis

What is it?

Lynis is an open-source security auditing tool from CISOfy. It runs a few hundred checks on a Linux system (SSH settings, firewall, updates, kernel settings, file permissions, logging, ...) and produces: - a **hardening index** from 0 to 100 - **warnings** (serious findings) and **suggestions** (improvements), each with a test id like `SSH-7408` - a machine-readable report, `/var/log/lynis-report.dat`

It only **reads**; it never changes the system.

Why this project uses it

"We hardened the servers" is an opinion. **"The Lynis hardening index went from X to Y, and these suggestions were resolved"** is a measurement made by an independent tool. Lynis gives: 1. **Before/after proof** of what the baseline achieves (see `docs/test-results.md` for the measured values). 2. **Continuous checking**: a weekly audit on every server, with the score sent to Prometheus, so a drop raises an alert.

Alternatives: **OpenSCAP** with CIS or DISA STIG profiles (formal compliance reports, heavier), **CIS-CAT** (commercial), cloud tools such as AWS Inspector. Lynis is light, needs no agent or server, and its score is easy to explain.

Honest limits: the index is a summary, not a guarantee. Some tests are skipped inside containers, and a few suggestions are deliberately not followed (for example moving SSH to another port). Compare scores from the **same Lynis version on the same kind of machine**, which is why the version is pinned.

How it works in this project

```
roles/lynis
├─ downloads Lynis 3.1.7 from the official CISOfy GitHub tag, SHA-256 verified
├─ unpacks it to /opt/lynis-3.1.7 (owned by root: Lynis refuses files owned by others)
├─ installs /usr/local/sbin/server-baseline-audit:
│   lynis audit system --quick --cronjob → /var/log/lynis-report.dat
│   lynis_report.py prom                 → /var/lib/node_exporter/textfile/lynis.prom
└─ server-baseline-audit.timer: weekly, random delay up to 1h, catches up after downtime
```

The report converter: `lynis_report.py`

`roles/lynis/files/lynis_report.py` is a small Python program (standard library only, so it runs on any server that runs Ansible). It has two commands:

- `prom REPORT --output FILE` writes metrics for Prometheus: `lynis_hardening_index 0..100`
`lynis_warnings / lynis_suggestions / lynis_tests_done lynis_last_run_timestamp_seconds`
`lynis_info{version="3.1.7"} 1` It writes to a temporary file and **renames** it, so `node_exporter` never reads a half-written file.
- `compare BEFORE AFTER` prints the before/after Markdown table and lists the suggestion ids that were resolved.

It has its own unit tests: `tests/test_lynis_report.py`. One of them caught a real bug during development: Python's `%g` number format printed the timestamp as `1.79102e+09`, losing precision.

Before and after

`playbooks/audit.yml` installs Lynis (if missing), runs the audit now, and copies each report to the controller as `reports/lynis/<host>-<label>.dat` :

```
scripts/audit.sh before      # on the fresh servers
make harden
scripts/audit.sh after
scripts/audit.sh compare     # the table
```

Why GitHub and not [downloads.cisofy.com](https://cisofy.com/downloads)?

The CISOfy download server answers **HTTP 403** to Ansible's HTTP client (it filters by user agent). Instead of pretending to be a browser, the role downloads the same release from CISOfy's official GitHub tag and pins its checksum:

```
lynis_version: 3.1.7
lynis_checksum: sha256:48d829d0dc2c583a3e838cc09a7190b69a3af844bcb913c7cf9c0226b04b95c5
lynis_url: https://github.com/CISOfy/lynis/archive/refs/tags/{{ lynis_version }}.tar.gz
```

Where it is integrated

- `roles/lynis/` : install, timer, wrapper, converter
- `roles/lynis/tasks/audit.yml` : "run now and fetch"
- `lab/prometheus/rules/server-baseline.yml` : `LynisScoreLow` (below 75 for 15 min) and `LynisReportStale` (no audit for 8 days)
- Grafana: "Lynis hardening index" bar gauge and "Days since last audit"

Try it

```
docker compose -f lab/compose.yaml exec web-01 /usr/local/sbin/server-baseline-audit # about a minute
docker compose -f lab/compose.yaml exec web-01 grep -E '^(hardening_index|lynis_tests_done)='
/var/log/lynis-report.dat
docker compose -f lab/compose.yaml exec web-01 sh -c "grep '^suggestion' /var/log/lynis-report.dat |
head"
docker compose -f lab/compose.yaml exec web-01 cat /var/lib/node_exporter/textfile/lynis.prom
docker compose -f lab/compose.yaml exec web-01 systemctl list-timers server-baseline-audit.timer
```

Common mistakes

- **Chasing 100.** Some suggestions don't fit every server. Decide, document, and set a realistic target (here: 75 and above).
- **Comparing scores across Lynis versions or machine types.** Pin the version.
- **Running it once.** Security decays; the weekly timer and the stale-report alert keep it honest.
- **Using the distribution's old package** without noticing it is several versions behind.

Check yourself

1. Does Lynis change anything on the server?

2. Why is the metrics file written to a temporary file first and then renamed?
3. Which alert tells you the weekly audit stopped running?

▼ **Answers**

1. No. It only reads and reports; the Ansible roles make the changes.
2. A rename is atomic: node_exporter sees either the old complete file or the new complete file, never a half-written one (which would be a scrape error).
3. `LynisReportStale: time() - lynis_last_run_timestamp_seconds` is above 8 days.

10. Prometheus and node_exporter

What is it?

- **Prometheus** collects numbers over time (**time series**) by visiting a `/metrics` web page on each target every few seconds (**scraping**). It stores them, lets you **query** them with PromQL, and evaluates **alert rules**.
- **node_exporter** is the official program that publishes a Linux server's numbers (CPU, memory, disk, network, ...) on port 9100.

```
web-01: node_exporter :9100/metrics ← every 15 s — Prometheus → alert rules → Grafana
node_cpu_seconds_total{cpu="0",mode="idle"} 1234.5
node_systemd_unit_state{name="ssh.service",state="active",type="notify"} 1
```

Why this project uses them

Hardening once is not enough; you need to **know** it is still in place. Prometheus answers, every 15 seconds: - Is every server reachable? (`up`) - Is every protection running? (SSH, fail2ban, firewall, update timer, audit timer, auditd on real machines) - What is each server's Lynis score, and how old is it? - Is someone being banned for guessing SSH logins?

Alternatives: Zabbix, Nagios/Icinga (classic server monitoring), Datadog or New Relic (hosted, paid), the ELK stack for logs. Prometheus is the open-source standard and pairs naturally with Grafana.

How it works: three ways node_exporter gets numbers

1. **Built-in collectors:** CPU, memory, filesystems, network, ...
2. **The systemd collector** (`--collector.systemd`) reads the state of chosen services over D-Bus:
`node_systemd_unit_state{name="ssh.service",state="active"} 1` . That's why the `common` role installs and starts `dbus` .
3. **The textfile collector** (`--collector.textfile.directory`) publishes any `*.prom` file in a folder. This is how **non-exporter tools report metrics**: - `fail2ban.prom` from the fail2ban timer (chapter 5) - `lynis.prom` from the weekly audit (chapter 9) - `baseline.prom` :
`server_baseline_info{kind="machine|container",os="...",version="1.0.0"} 1`

Where it is integrated

node_exporter, installed safely

`roles/node_exporter/tasks/main.yml` : - downloads version **1.12.1** with `get_url` and `checksum: sha256:...` (from the official `sha256sums.txt`), **before** unpacking - unpacks it into a versioned folder and points `/usr/local/bin/node_exporter` at it (easy upgrade and rollback) - runs it as the unprivileged `node_exporter` user, with systemd sandboxing (`node_exporter.service.j2`):

```
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
CapabilityBoundingSet=
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX AF_NETLINK
```

Port 9100 is open only to Prometheus (chapter 4).

Prometheus configuration

`lab/prometheus/prometheus.yml` scrapes both servers and turns `web-01:9100` into a clean `instance="web-01"` label with a **relabel rule**:

```
relabel_configs:
- source_labels: [__address__]
  regex: "([^:]+):\\d+"
  target_label: instance
  replacement: "$1"
```

Alert rules

`lab/prometheus/rules/server-baseline.yml` :

Alert	Fires when	Severity
ServerDown	<code>up{job="node"} == 0</code> for 2m	critical
SecurityServiceNotRunning	SSH, fail2ban, firewall, update timer or audit timer not active for 2m	critical
AuditdNotRunning	auditd not active on a real machine (and <code>on(instance)</code> <code>server_baseline_info{kind="machine"}</code>)	critical
LynisScoreLow	<code>lynis_hardening_index < 75</code> for 15m	warning
LynisReportStale	no audit for 8 days	warning
SSHAttackerBanned	<code>fail2ban_banned_current > 0</code>	info
Fail2banNotAnswering	<code>fail2ban_up == 0</code> for 5m	warning
MetricsFileBroken	<code>node_textfile_scrape_error == 1</code> for 10m	warning
DiskAlmostFull	less than 10% free for 10m	warning

`for:` means "the condition must stay true this long before the alert fires", which filters out short blips. The `and on(instance)` trick joins two metrics by server: auditd is only expected where the server says it is a machine, not a container.

Alert rules have unit tests

`tests/prometheus/server-baseline.test.yml` feeds **synthetic series** to `promtool test rules` and asserts which alerts fire, when, and with which text. Example: the firewall service goes down at minute 2; at minute 3 no alert (still inside `for: 2m`), at minute 5 the alert fires.

A lesson from writing these tests: series given every **1 hour** go "stale" between samples (Prometheus looks back only 5 minutes), which kept resetting the `for:` timer. The stale-report test uses 5-minute samples.

Try it

```
# Open http://localhost:9090 → Status → Targets (both servers UP), then Alerts.
# Queries to paste into the Graph tab:
#   up{job="node"}
#   node_systemd_unit_state{state="active"}
#   lynis_hardening_index
#   fail2ban_banned_current
#   server_baseline_info
docker compose -f lab/compose.yaml exec web-01 curl -s localhost:9100/metrics | grep -E
'^ (lynis|fail2ban|server_baseline)'
make promtool          # check the rules and run their unit tests
```

Common mistakes

- **Exposing port 9100 to the world.** Metrics reveal software versions and usage. Restrict it with the firewall.
- **Writing textfile metrics directly** to the final file: a half-written file causes scrape errors. Write and rename.
- **Alerts without** `for:` page on every blip.
- **Untested alert rules:** you find out they are wrong during an incident. Use `promtool test rules`.

Check yourself

1. How does a shell script get its numbers into Prometheus without being an exporter?
2. Why is the auditd alert joined with `server_baseline_info{kind="machine"}`?
3. What does the relabel rule change, and why?

▼ Answers

1. It writes a `*.prom` file to node_exporter's textfile directory; node_exporter publishes it with its own metrics.
2. auditd cannot run in containers. Without the join, the alert would fire forever on every lab server; with it, it fires only where auditd is expected.
3. It removes `:9100` from the `instance` label, so dashboards and alerts say `web-01` instead of `web-01:9100`.

11. Grafana

What is it?

Grafana draws dashboards from data sources such as Prometheus. Each **panel** runs a query and shows the result as a number, a gauge, a table or a chart.

Why this project uses it

Prometheus holds the numbers; Grafana makes them **readable at a glance**: "are all servers protected right now, and what is their score?" It is the screen you would put on the wall of an operations team.

Alternatives: Prometheus' own graph page (fine for queries, not for dashboards), Kibana, Datadog dashboards. Grafana is the standard companion of Prometheus.

How it works: provisioning as code

Nothing is clicked together by hand. At start-up, Grafana reads: - **Data sources** from

`lab/grafana/provisioning/datasources/datasources.yaml` :

```
datasources:
- name: Prometheus
  uid: prometheus
  type: prometheus
  url: http://prometheus:9090
  isDefault: true
  editable: false
```

- **Dashboards** from JSON files, via `lab/grafana/provisioning/dashboards/dashboards.yaml` , which points at `/var/lib/grafana/dashboards` . The dashboard is `lab/grafana/dashboards/server-baseline.json` and is set as the home page.

Because the dashboard is a file in Git, it is reviewed and versioned like code; CI checks it is valid JSON.

Access in the lab (`lab/compose.yaml`): anonymous users get the **Viewer** role; the admin password is generated by `scripts/lab-up.sh` into `.lab/grafana-admin-password` (git-ignored) and passed to Grafana as a Docker secret (`GF_SECURITY_ADMIN_PASSWORD__FILE`), never written into the compose file. Ports are bound to `127.0.0.1` only.

The dashboard: "Server baseline: security & health"

Row	Panel	Query (simplified)	Reads as
Security posture	Servers reporting	<code>sum(up{job="node"})</code>	how many servers Prometheus can scrape
	Lynis hardening index	<code>lynis_hardening_index</code>	bar per server; red < 60, orange < 75, green ≥ 75

Row	Panel	Query (simplified)	Reads as
	Addresses banned now	<code>sum(fail2ban_banned_current)</code>	orange as soon as anyone is banned
	Firing alerts	<code>count(ALERTS{alertstate="firing"})</code> or <code>vector(0)</code>	<code>or vector(0)</code> shows 0 instead of "no data"
	Days since last audit	<code>max((time() - lynis_last_run_timestamp_seconds) / 86400)</code>	orange after 7, red after 8 days
	Security services	<code>node_systemd_unit_state{state="active"}</code> as a server × service grid	green 1 = running, red 0 = stopped
	Baseline version	<code>server_baseline_info</code> as a table	which baseline version and OS each server runs
Attacks	fail2ban: SSH failures and bans	<code>fail2ban_failed_current</code> , <code>fail2ban_banned_current</code>	the brute-force test shows up here
Resources	CPU busy, Memory used, Network received	<code>node_exporter</code> basics	is the server healthy

Try it

1. Open <http://localhost:3000>. The dashboard is the home page.
2. Run the brute-force exercise from chapter 5 and watch "Addresses banned now" and the fail2ban chart (the textfile timer runs every minute, so allow 1–2 minutes).
3. Stop a protection and watch the "Security services" grid turn red: `bash docker compose -f lab/compose.yaml exec db-01 systemctl stop fail2ban` # after ~2 minutes `SecurityServiceNotRunning` also fires in Prometheus (Alerts tab) make `hardened` # starts it again

Common mistakes

- **Building dashboards by clicking** and never exporting them: they are lost with the container.
- **Panels that show "No data" instead of 0:** use `or vector(0)` where 0 is the honest answer.
- **Default** `admin/admin` **password**, or a password committed to Git. Use a generated secret.

Check yourself

1. Where does Grafana get the dashboard from when the container starts?
2. Why does the "Firing alerts" query end with `or vector(0)` ?
3. How is the Grafana admin password kept out of the repository?

▼ Answers

1. From the provisioning config, which loads the JSON file `lab/grafana/dashboards/server-baseline.json` mounted read-only into the container.
2. When nothing is firing, `count(...)` returns no data. `or vector(0)` turns that into a clear 0.

3. `scripts/lab-up.sh` generates it into `.lab/grafana-admin-password` (git-ignored), and compose passes it as a Docker secret that Grafana reads through `GF_SECURITY_ADMIN_PASSWORD__FILE`.

12. Molecule and Docker

What is it?

- **Docker** runs programs in **containers**: isolated processes with their own filesystem, started from an **image**. Here, containers play the role of servers.
- **Molecule** is the testing framework for Ansible. It creates throwaway machines, applies your roles, runs them a second time to prove idempotence, runs your checks, and deletes the machines.

Why this project uses them

A hardening change that works on Ubuntu can break on Debian (different package names, different SSH start-up, a file that doesn't exist). Molecule tests **every role on three distributions** on every change, at no cloud cost:

Platform	Image
Ubuntu 24.04	geerlingguy/docker-ubuntu2404-ansible
Debian 12	geerlingguy/docker-debian12-ansible
Debian 13	geerlingguy/docker-debian13-ansible

These images are widely used for Ansible testing because they start **systemd** inside the container, so services and timers behave like on a real server.

Alternatives: Vagrant with virtual machines (closer to reality, much slower), cloud VMs (real, but cost money and time), Testinfra or Goss for the checks. Docker is fast and free; the parts containers cannot test (auditd, global kernel settings) are covered by the CI `vm` job instead.

How it works: the scenario lifecycle

`molecule/default/molecule.yml` defines the test sequence:

```
scenario:
  test_sequence:
    - destroy      # remove leftovers from an earlier run
    - syntax       # ansible-playbook --syntax-check
    - create       # start the three containers
    - converge     # apply the baseline (molecule/default/converge.yml → playbooks/site.yml)
    - idempotence  # apply again: ANY "changed" task fails the test
    - verify       # molecule/default/verify.yml → playbooks/verify.yml
    - destroy
```

`converge.yml` and `verify.yml` simply import the real playbooks, so **Molecule tests exactly what production runs**:

```
- name: Apply the baseline
  ansible.builtin.import_playbook: ../../playbooks/site.yml
```

Why the containers are privileged

```
privileged: true
cgroupns_mode: host
volumes:
  - /sys/fs/cgroup:/sys/fs/cgroup:rw
command: "" # keep the image's default start command: systemd
```

systemd needs access to the host's control groups (cgroups) to manage services, and the firewall needs to create nftables rules in the container's network namespace. **Privileged containers are a test-lab pattern, not a production one:** they have almost all of the host's capabilities.

Test data without secrets

The admin key in `molecule.yml` is a real public key whose private half was thrown away when it was created. It only has to exist, because the lockout guard requires an admin with a key; nobody logs in with it. Molecule connects with `docker exec`, not SSH.

serial in Molecule

`site.yml` hardens one server at a time. Molecule runs all three at once by passing an extra variable, because `serial` is a play keyword that cannot come from the inventory:

```
options:
  extra-vars: baseline_serial=0
```

The lab is Docker too

The lab (`lab/compose.yaml`) uses the same idea, but over **real SSH**:

Container	Address	Role
controller	172.30.0.5	Ansible + tools, SSH key in a Docker volume
web-01	172.30.0.21	Ubuntu 24.04 server, distribution defaults (<code>lab/target/Dockerfile</code>)
db-01	172.30.0.22	Debian 12 server
attacker	172.30.0.66	brute force and port probes in the tests
prometheus	172.30.0.10	the only address allowed to reach port 9100
grafana	172.30.0.11	dashboard on <code>localhost:3000</code>

Each server generates its **own SSH host keys at first boot** (a drop-in for `ssh.service`, `lab/target/ssh-hostkeys.conf`, runs `ssh-keygen -A` just before `sshd` starts), because keys baked into an image would be identical on every container. A first version used a separate unit and lost a race against Ubuntu's early `ssh.socket`. The story is in `docs/troubleshooting.md`.

Try it

```
make molecule                # the full test sequence; or step by step:
MSYS_NO_PATHCONV=1 docker run --rm -it -v "$PWD:/work" -v /var/run/docker.sock:/var/run/docker.sock \
server-baseline-controller:dev bash
#   inside: molecule create; molecule converge; molecule idempotence; molecule verify
#           molecule login -h debian12      # a shell in one test container
#           molecule destroy
```

Common mistakes

- **Testing only `converge`** . The idempotence step finds the bugs that make re-runs noisy and drift checks useless.
- **Starting the container with `bash` as command** instead of `systemd`: every service task fails.
- **Expecting everything to work in containers:** `auditd` and global kernel settings need a VM.
- **Editing a role while Molecule runs:** role defaults are read at play start, so a mid-run change gives confusing errors.

Check yourself

1. Which Molecule step fails if a task reports "changed" every time?
2. Why do `converge.yml` and `verify.yml` import `playbooks/site.yml` and `playbooks/verify.yml` instead of having their own tasks?
3. Name two things this project cannot test in containers, and where they are tested instead.

▼ Answers

1. **idempotence:** it runs the playbook a second time and fails on any changed task.
2. So the test runs exactly the code production runs. Separate test playbooks could pass while the real one is broken.
3. `auditd` (kernel audit is host-wide) and global kernel settings (`kernel.*` , `fs.*`). Both are tested by the CI `vm` job on a real GitHub runner VM.

13. GitHub Actions

What is it?

GitHub Actions is GitHub's built-in automation. A **workflow** file in `.github/workflows/` says *when* to run (on push, on pull request, by hand) and *what* to run: **jobs** made of **steps**, each job on a fresh virtual machine called a **runner**.

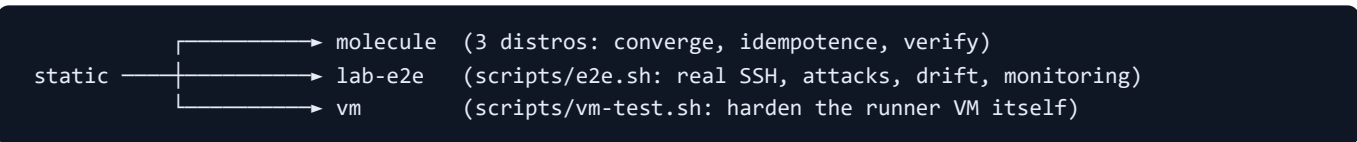
Why this project uses it

Every change to a security baseline must be proven not to break servers. CI runs **all** tests on every push and pull request, so a mistake is caught before anyone applies it.

Alternatives: GitLab CI, Jenkins, CircleCI. GitHub Actions is built into the place where the code lives and is free for public repositories.

How it works: the four jobs

`.github/workflows/ci.yaml` :



Job	What it runs	Why
static	<code>scripts/lint.sh</code> (yamllint, ansible-lint production profile, syntax check, ShellCheck, pytest), <code>promtool check config</code> + <code>promtool test rules</code> , <code>docker compose config</code> , JSON check of the dashboard	Fast feedback; the other jobs only start if it passes (<code>needs: static</code>)
molecule	<code>molecule test</code>	Roles work and are idempotent on Ubuntu 24.04, Debian 12, Debian 13
lab-e2e	<code>scripts/e2e.sh</code>	The full story over real SSH: before/after Lynis, idempotence, access tests, brute force, drift, safety nets, metrics. Uploads <code>reports/</code> as an artifact
vm	<code>scripts/vm-test.sh</code>	auditd and every kernel setting on a real kernel; Lynis before/after on a VM

The VM job: hardening the runner

A GitHub runner is a **throwaway virtual machine**, deleted after the job. That makes it a free, real Linux VM to test what containers cannot. `scripts/vm-test.sh` applies the baseline to `localhost` with `inventories/local/hosts.yaml` (`ansible_connection: local`), checks idempotence, runs the verify playbook, proves auditd records a change, and writes the results to the job summary.

Because it hardens the machine it runs on, the script refuses to run outside CI unless you pass `--yes-harden-this-machine` . The admin key it needs is generated per run and deleted afterwards.

Details worth knowing

- `permissions: contents: read` : the workflow token can only read the repository (least privilege).
- `concurrency: cancel-in-progress` : a new push cancels the old run of the same branch.
- Tools are installed from the **pinned** `requirements.txt` and `requirements.yml`, the same versions as the local controller image.
- `if: always()` on the report upload and `make lab-down` steps: they run even when the test failed, which is exactly when you need the logs.

Try it

Run the same checks locally:

```
make lint           # static checks inside the controller image
make promtool
make molecule
make e2e            # builds a fresh lab and runs the full end-to-end test
```

On GitHub: open the repository's **Actions** tab, pick a run, and open the **Summary**: the e2e and VM jobs write their results there.

Common mistakes

- **CI that only lints.** Linting proves the YAML is valid, not that SSH still lets you in.
- **Different tool versions locally and in CI.** Pin them in one place.
- **Running a "harden this machine" script on your laptop.** The guard flag exists for this.
- **Not keeping logs of failed runs:** upload artifacts with `if: always()`.

Check yourself

1. Why do the molecule, lab-e2e and vm jobs have `needs: static` ?
2. Why is it safe to harden the CI runner, but not your laptop?
3. Which job proves that auditd really records events?

▼ Answers

1. So a simple lint error fails fast and cheaply, without starting the long jobs.
2. The runner is a throwaway VM deleted after the job. Your laptop would keep the firewall, SSH and kernel changes.
3. The **vm** job (`scripts/vm-test.sh`): it changes `/etc/hosts` and checks `ausearch -k network` finds the event.

14. How everything fits together

This chapter follows **one** `make harden` **run** and then **one SSH attack** through every tool, so you can see how each piece hands work to the next.

A. The life of one `make harden`

```
make harden
└─ scripts/harden.sh → docker compose exec controller ansible-playbook playbooks/site.yml
   (ansible.cfg: inventory inventories/lab/hosts.yml, SSH key /keys/id_ed25519, pipelining)

Gathering facts on web-01 (over SSH as ops, sudo)                                distribution, virtualization type
...
serial: 1 → web-01 first; db-01 only starts if web-01 succeeded

common                                assert Debian/Ubuntu · LOCKOUT GUARD: an admin with a key exists
apt: base packages in, telnet & co. out · dbus started · timezone
group ssh-users · user ops · authorized_keys (exclusive)
sudoers — validate: visudo -cf → installed
banners · UMASK 027 · no core dumps · journald persistent → notify Restart journald
kernel_hardening /etc/sysctl.d/90-server-baseline.conf · /etc/modprobe.d blocklist
read live values → container? only net.* · write only differences
ssh_hardening LOCKOUT GUARD: ssh-users has members
Include line in sshd_config · /run/sshd
00-server-baseline.conf — validate: sshd -t → installed → notify Reload sshd
weak moduli removed · Ubuntu: ssh.socket off, ssh.service on
firewall LOCKOUT GUARD: a rule for port 22 exists
/etc/nftables.d/server-baseline.nft — validate: nft -c → installed → notify
Reload firewall server-baseline-firewall.service started + enabled
kernel check: table loaded? (catches hand-deleted rules)
fail2ban jail.d/server-baseline.local → notify Restart fail2ban · metrics timer
auto_updates 20auto-upgrades + 52server-baseline · apt timers enabled
auditd rules.d/50-server-baseline.rules · container? install only : load + start
node_exporter get_url with sha256 → unpack → symlink · baseline.prom · sandboxed unit → notify
Restart
lynis get_url with sha256 → /opt/lynis-3.1.7 · audit wrapper · weekly timer

post_tasks flush_handlers: Reload sshd, Reload firewall, Restart fail2ban ... run NOW
reset_connection + wait_for_connection: a NEW SSH login must still work
→ same for db-01
PLAY RECAP: web-01 changed=N, db-01 changed=N (second run: changed=0 → idempotent)
```

Three kinds of safety net appear in that trace: 1. **Guards** (`assert`) stop the run *before* anything that could lock you out. 2. `validate`: stops a broken file from ever being installed. 3. **The final reconnect** proves access still works, server by server, before moving to the next one.

B. The life of one SSH brute-force attack

```

t=0    attacker (172.30.0.66): ssh guess1@db-01 ... guess8@db-01
      └─ TCP 22 allowed by the firewall rule "ssh" (nftables, table
server_baseline)
t=0    sshd: "Invalid user guess1 from 172.30.0.66" ... refuses (publickey only) (sshd, AllowGroups, keys
only)
      └─ message goes to the systemd journal (journal, persistent)
t≈s    fail2ban sshd jail (backend = systemd, mode = aggressive) counts 5 failures in 10 min
      └─ ban: 172.30.0.66 added to table inet f2b-table (fail2ban → nftables)
      └─ further connections from the attacker: rejected (nftables)
      └─ the controller 172.30.0.5 is in ignoreip: Ansible still connects
≤1 min fail2ban-textfile.timer → fail2ban_banned_current{jail="sshd"} 1 (textfile collector file)
+15 s  Prometheus scrapes db-01:9100 (allowed: source 172.30.0.10) (node_exporter, firewall
rule)
+15 s  rule SSHAttackerBanned: fail2ban_banned_current > 0 → FIRING (Prometheus alert rules)
      Grafana: "Addresses banned now" = 1, fail2ban chart shows the ban (Grafana)
+1h    ban expires; a repeat offender gets 2h, then 4h ... (bantime.increment)

```

The measured times for this attack in the lab are in [docs/test-results.md](#).

C. Remove one piece: what breaks?

Remove	What happens
The <code>common</code> lockout guard	An empty admin list would let SSH become key-only with nobody allowed in
<code>validate: sshd -t</code>	One wrong cipher name, a reload, and sshd may refuse every new login
<code>00-</code> prefix of the SSH drop-in	A cloud-init drop-in could switch password login back on
The firewall's own table (using <code>flush ruleset</code>)	fail2ban's bans and Docker's rules are wiped on every reload
The firewall's kernel check	Someone deletes the rules by hand; the service still says "active"; the drift check misses it
<code>ignoreip</code> for the controller	One failed test locks Ansible out of the server
<code>backend = systemd</code>	fail2ban looks for <code>/var/log/auth.log</code> , which Debian doesn't write; nothing is ever banned
<code>check_mode: false</code> on read tasks	The drift check can't see kernel drift
<code>serial: 1</code>	A bad change breaks every server at once
Textfile metrics	Prometheus can't see bans or Lynix scores; no alerts for them
The <code>kind</code> label on <code>server_baseline_info</code>	<code>AuditdNotRunning</code> fires forever on every container
Molecule's idempotence step	A task that always "changes" goes unnoticed and drift checks become noise
The CI <code>vm</code> job	auditd and global kernel settings would never be tested

D. Concept → file map

Concept	File
Which servers, which variables	<code>inventories/lab/hosts.yml</code> , <code>inventories/lab/group_vars/baseline.yml</code>
Order of the roles, rolling update, reconnect test	<code>playbooks/site.yml</code>
All checks in one place	<code>playbooks/verify.yml</code> + <code>roles/*/tasks/verify.yml</code>
Before/after audit	<code>playbooks/audit.yml</code> , <code>roles/lynis/tasks/audit.yml</code>
SSH settings	<code>roles/ssh_hardening/templates/sshd-baseline.conf.j2</code>
Firewall rules	<code>roles/firewall/templates/server-baseline.nft.j2</code>
fail2ban jail	<code>roles/fail2ban/templates/jail.local.j2</code>
Kernel settings	<code>roles/kernel_hardening/defaults/main.yml</code>
Audit rules	<code>roles/auditd/templates/server-baseline.rules.j2</code>
Lynis → metrics	<code>roles/lynis/files/lynis_report.py</code>
Alerts and their tests	<code>lab/prometheus/rules/server-baseline.yml</code> , <code>tests/prometheus/server-baseline.test.yml</code>
Dashboard	<code>lab/grafana/dashboards/server-baseline.json</code>
Lab layout	<code>lab/compose.yaml</code>
Drift check	<code>scripts/drift-check.sh</code>
End-to-end test	<code>scripts/e2e.sh</code>
Role tests	<code>molecule/default/molecule.yml</code>
CI	<code>.github/workflows/ci.yaml</code>

15. Hands-on labs

Do these in order. Each lab says what to do, what you should see, and what you learned. Commands run from the repository root in bash (Git Bash on Windows). A shortcut used below:

```
lab() { docker compose -f lab/compose.yaml exec "$@"; }      # lab web-01 <command>
```

Open three windows: a terminal, Prometheus (<http://localhost:9090>) and Grafana (<http://localhost:3000>).

Lab 1: Explore a fresh server (15 min)

```
make lab-up
lab web-01 bash -c '. /etc/os-release; echo $PRETTY_NAME'
lab web-01 sh -c "sshd -T | grep -E 'passwordauthentication|permitrootlogin'"
lab web-01 nft list ruleset                                # empty: no firewall
lab web-01 systemctl is-active fail2ban                    # not installed
lab attacker ssh -o BatchMode=yes -o StrictHostKeyChecking=no -o PreferredAuthentications=none
probe@web-01
```

You should see: password login on, root login with a key allowed (`prohibit-password`), no firewall, Permission denied (`publickey,password`).

✓ **You learned:** what "distribution defaults" really look like.

Lab 2: Measure, harden, measure again (20 min)

```
scripts/audit.sh before
make harden                # watch the roles run, web-01 first, then db-01
make harden                # again: every host shows changed=0
make verify
scripts/audit.sh after
scripts/audit.sh compare   # the before/after table and the resolved Lynis suggestions
```

Repeat the Lab 1 commands and compare. In Grafana, the Lynis bars and the "Security services" grid fill in.

✓ **You learned:** idempotence in practice, and how to prove an improvement with an independent score.

Lab 3: Break it: drift (15 min)

```
lab web-01 sed -i 's/^PasswordAuthentication no/PasswordAuthentication yes/' /etc/ssh/sshd_config.d/00-
server-baseline.conf
lab web-01 nft delete table inet server_baseline
lab web-01 sysctl -w net.ipv4.conf.all.accept_redirects=1
make drift; echo "exit code: $?"
```

You should see: exit code 2, a diff of the SSH file, and "changed" for the firewall and the kernel setting. Check that nothing was repaired yet: `lab web-01 nft list tables`. Then:

```
make harden
make drift                # "No drift", exit code 0
```

✅ **You learned:** check mode reports without changing, and why the firewall role asks the kernel instead of trusting the service state.

Lab 4: Break it: the safety nets (15 min)

Each of these must **fail before changing anything**:

```
# 1. No administrator → lockout guard
scripts/harden.sh --limit web-01 -e '{"common_admin_users": []}'
# 2. Impossible firewall address → nft -c rejects it
scripts/harden.sh --limit web-01 --tags firewall \
  -e '{"firewall_rules": [{"name": "ssh", "port": 22, "sources_v4": ["0.0.0.0/0"], "sources_v6": []},
{"name": "typo", "port": 9100, "sources_v4": ["300.1.1.1/32"], "sources_v6": []}]}'
# 3. Unknown cipher → sshd -t rejects it
scripts/harden.sh --limit web-01 --tags ssh -e '{"ssh_hardening_ciphers": ["no-such-cipher"]}'
# Afterwards: nothing changed and you can still log in
make drift
```

Read the error messages: "lock everyone out", "failed to validate".

✅ **You learned:** guards and `validate:` turn dangerous mistakes into harmless failed runs.

Lab 5: Attack: SSH brute force (15 min)

```
for i in 1 2 3 4 5 6 7 8; do
  docker compose -f lab/compose.yaml exec -T attacker ssh -o BatchMode=yes -o StrictHostKeyChecking=no -o ConnectTimeout=5 guess$i@db-01 true
done
lab db-01 fail2ban-client status sshd # 172.30.0.66 in the banned list
lab db-01 nft list table inet f2b-table
lab attacker ssh -o ConnectTimeout=5 probe@db-01 # refused now
lab db-01 journalctl -u ssh --since "5 min ago" | grep -i invalid | tail -3
```

Within about two minutes: Prometheus → Alerts shows `SSHAttackerBanned` firing; Grafana shows "Addresses banned now = 1". Unban:

```
lab db-01 fail2ban-client set sshd unbanip 172.30.0.66
```

✅ **You learned:** the full path from a log line to a firewall rule to an alert (chapter 14, part B).

Lab 6: Change a parameter (20 min)

a) Stricter fail2ban. In `inventories/lab/group_vars/baseline.yml` add `fail2ban_maxretry: 3`, run `make harden`, and check only the fail2ban task and its handler changed. Repeat Lab 5: the ban now comes after 3 attempts.

b) Open a new port. Add a rule to `firewall_rules`:

```
- name: web
  port: 80
  sources_v4: ["0.0.0.0/0"]
  sources_v6: [":::/0"]
```

`make harden`, then `lab web-01 nft list table inet server_baseline` shows `comment "web"`. The verify playbook now also checks this rule exists. Remove it again afterwards.

✅ **You learned:** the baseline is data-driven: inventory variables change behaviour, the roles stay the same.

Lab 7: Stop a protection and watch the alert (10 min)

```
lab web-01 systemctl stop fail2ban
```

Grafana's "Security services" grid turns red for web-01; after 2 minutes `SecurityServiceNotRunning` fires in Prometheus. `make harden` starts it again (the role's `state: started`), and the alert resolves.

✅ **You learned:** monitoring the protections themselves, not just CPU and memory.

Lab 8: Test the tests (30 min)

A test you have never seen fail proves nothing. Make each test layer fail on purpose, then undo the change (`git checkout -- <file>`).

1. **Idempotence:** add this task at the end of `roles/common/tasks/main.yml`: `yaml - name: A task that always changes ansible.builtin.command: date` `make molecule` fails at the **idempotence** step (and `ansible-lint` complains with `no-changed-when` in `make lint`).
2. **Verify:** in `roles/ssh_hardening/templates/sshd-baseline.conf.j2` change `MaxAuthTries {{ ssh_hardening_max_auth_tries }}` to `MaxAuthTries 6`. Molecule's **verify** step fails on `maxauthtries`.
3. **Alert tests:** in `lab/prometheus/rules/server-baseline.yml` change `for: 2m` of `SecurityServiceNotRunning` to `for: 10m`. `make promtool` fails: the test expects the alert at minute 5.
4. **Unit tests:** in `roles/lynis/files/lynis_report.py` in `to_prometheus`, change the `lynis_warnings` line to count `report.suggestions` instead of `report.warnings`. `pytest tests` (or `make lint`) fails on `lynis_warnings 1`.

✅ **You learned:** what each test layer protects, and that each one really catches the mistake it is meant for.

Lab 9 (optional): The full end-to-end test

```
make e2e
cat reports/e2e-results.md
```

This rebuilds the lab from zero and runs Labs 1–7 automatically, with real measurements. Compare your numbers with `docs/test-results.md`.

When you are done: `make lab-down`.

Glossary

Term	Meaning
Agentless	Ansible needs nothing installed on the servers except SSH and Python
Alert rule	A Prometheus expression that "fires" when true for a set time (<code>for:</code>)
auditd / ausearch	The service that writes kernel audit records / the tool that searches them by key
audit	Audit user id: the id of the person who logged in, kept even after <code>sudo</code>
Bantime / findtime / maxretry	fail2ban: ban length / counting window / failures allowed in that window
Baseline	The minimum security configuration every server must have
Become	Ansible's word for running a task with sudo
Check mode (<code>--check</code>)	Ansible reports what it would change without changing anything
Collector (node_exporter)	A part of node_exporter that produces one group of metrics (cpu, systemd, textfile, ...)
Connection tracking (<code>ct state</code>)	The kernel remembers connections so replies can be allowed automatically
Container	An isolated process with its own filesystem and network, sharing the host kernel
Control node / controller	The machine that runs Ansible; here the <code>controller</code> container
Default-deny	Firewall policy: drop everything that no rule explicitly allows
Diff mode (<code>--diff</code>)	Ansible shows the before/after of changed files
Drift	A server's configuration differing from the baseline, usually through manual changes
Drop-in	A small config file in a <code>.d/</code> folder that adds to or overrides the main file
Facts	Information Ansible gathers about a server before running tasks
fail2ban jail	A log source + filter + ban action, here for sshd
Handler	An Ansible task that runs only when notified by a changed task, once, at the end
Hardening	Reducing a system's attack surface: fewer ways in, stronger locks, better records
Hardening index	Lynis' summary score from 0 to 100
Hook (nftables)	The point in the kernel's packet path where a chain is attached (input, forward, output)
Idempotent	Running it again gives the same result and changes nothing if nothing drifted
Inventory	Ansible's list of servers, groups and their variables
Jinja2	The template language Ansible uses: <code>{{ variable }}</code> , <code>{% for %}</code>
Journal (journalctl)	systemd's log store; read with <code>journalctl</code>
Key pair	A private key (kept secret) and a public key (put on servers) for SSH login

Term	Meaning
Least privilege	Give every user and program only the rights it needs
Lockout guard	An assert that stops the run before a change could lock administrators out
Lynis	Open-source security auditing tool that scores a Linux system
Molecule	Test framework for Ansible roles: create, converge, idempotence, verify, destroy
nftables / nft	The Linux kernel firewall / its command-line tool
node_exporter	Prometheus' official exporter of Linux server metrics, port 9100
Pipelining	Ansible setting that sends modules over the existing SSH session (faster)
Play / playbook	One "on these hosts, do these things" block / a file of plays
Privileged container	A container with almost all host capabilities; needed here for systemd in tests
PromQL	Prometheus' query language
promtool	Prometheus' tool to check configs and unit-test rules
Provisioning (Grafana)	Loading data sources and dashboards from files at start-up
Relabelling	Rewriting a target's labels in Prometheus before storing samples
Role	A reusable Ansible folder of tasks, defaults, templates and handlers for one job
Scrape	Prometheus fetching a target's <code>/metrics</code> page
Serial	Ansible rolling update: how many hosts to change at a time
sshd -t / sshd -T	Test the SSH server config / print the effective settings
Socket activation	systemd listens on a port and starts the service on the first connection (Ubuntu's <code>ssh.socket</code>)
sudo / sudoers	Run a command as root / the file that says who may
sysctl	Kernel settings under <code>/proc/sys</code> , persisted in <code>/etc/sysctl.d/</code>
systemd unit / service / timer	Something systemd manages / a long-running program / a schedule
Tags	Labels to run part of a playbook (<code>--tags ssh</code>)
Textfile collector	node_exporter feature that publishes <code>*.prom</code> files written by other tools
unattended-upgrades	Debian/Ubuntu tool that installs (security) updates automatically
validate:	Ansible option: test a new file with a command before installing it

Interview questions this project prepares you for

Try to answer each question out loud first, then open the answer.

Ansible

1. What does "idempotent" mean, and how do you prove a playbook is?

▼ Answer

Running it again gives the same result and changes nothing if nothing drifted. Proof: run it twice and require `changed=0` on the second run. Molecule's idempotence step, the lab end-to-end test and the CI VM test all do this here.

2. How do you make a `command` task idempotent?

▼ Answer

Read first (`changed_when: false` , `check_mode: false`), then run the write command only when the value differs, and set `changed_when` honestly. The `kernel_hardening` role does exactly this for `sysctl` values.

3. What is the difference between a task and a handler? When do handlers run?

▼ Answer

Handlers run only when notified by a task that reported "changed", once per play, after the tasks, or earlier at `meta: flush_handlers` . Used for reloads and restarts so services aren't restarted on every run.

4. How do you change hundreds of servers without breaking all of them at once?

▼ Answer

A rolling update: `serial` (here 1 server at a time) with `max_fail_percentage: 0` , so the first failure stops the run. Plus a reconnect test at the end of each server (`reset_connection` + `wait_for_connection`) so access problems are found on the first server.

5. How do you detect configuration drift with Ansible?

▼ Answer

Run the same playbook with `--check --diff` : it reports what would change without changing anything. `scripts/drift-check.sh` turns the recap into an exit code (0 = clean, 2 = drift). Read tasks need `check_mode: false` so the comparison still has data.

6. What does `validate:` do on the template module? Give examples.

▼ Answer

It runs a command against the new file (in a temporary path) and installs it only if the command succeeds: `visudo -cf %s` for sudoers, `sshd -t -f %s` for SSH, `nft -c -f %s` for the firewall. A broken file never reaches the server.

7. Where would you put variables, and why use role defaults?

▼ Answer

Role `defaults/main.yml` hold safe, documented defaults with the lowest priority; inventories (`group_vars`) override them per environment. Prefixing variables with the role name (`ssh_hardening_*`) avoids collisions and is checked by ansible-lint.

Linux security

8. Walk me through how you would harden a fresh Linux server.

▼ Answer

Admin users with keys and sudo; key-only SSH, no root login, allowed group, modern crypto; default-deny firewall with a short allow-list; fail2ban; automatic security updates; kernel sysctl hardening and blocked rare modules; auditd for accountability; remove legacy packages; persistent logs; then measure with Lynis and monitor that every protection keeps running.

9. Why disable password authentication and root login over SSH?

▼ Answer

Passwords can be guessed or reused; keys cannot be brute-forced. Root is the one account name every attacker knows, and direct root logins lose the information of which person acted. Admins log in as themselves and use sudo, which is logged.

10. How do you make sure an SSH hardening change doesn't lock you out?

▼ Answer

Guards before the change (an admin with a key exists; the allowed group has members; the firewall allows the SSH port), `sshd -t` validation, a reload instead of a stop, a fresh test connection afterwards, one server at a time, and the controller on fail2ban's ignore list.

11. How do you check the SSH settings that are really in effect?

▼ Answer

`sshd -T`. With drop-in files, the first value read wins, so reading `sshd_config` alone can mislead. The verify tasks assert on the `sshd -T` output.

12. Why does the firewall use its own nftables table instead of `flush ruleset` ?

▼ Answer

`flush ruleset` deletes every rule on the machine, including fail2ban's bans and Docker's networking rules. Replacing only its own table in one atomic transaction leaves the others untouched.

13. How does fail2ban work, and what are its limits?

▼ Answer

It reads logs (here the systemd journal), counts failures per address within `findtime`, and bans after `maxretry` by adding an nftables rule for `bantime`. Limits: distributed attacks from many addresses, and it is noise reduction, not the defence; key-only SSH is the defence.

14. Which kernel settings would you harden, and which would you leave alone?

▼ Answer

Harden: no ICMP redirects or source routing, reverse-path filtering, SYN cookies, `kptr_restrict`, `dmesg_restrict`, `ptrace_scope`, unprivileged BPF off, `suid_dumpable=0`, protected links. Leave alone: `ip_forward` on container hosts, which need it.

15. What is auditd for, and why can't you test it in a container?

▼ Answer

It records changes to important files and root commands at the kernel level, with the login user id (`audit`), for investigations and compliance. The kernel audit system is not namespaced; it belongs to the host, so containers get "Operation not permitted". This project tests it on a throwaway CI VM.

16. Should servers install updates automatically?

▼ Answer

Security updates: yes, daily, because most attacks use known, already-fixed vulnerabilities. Feature updates and reboots: planned. The baseline keeps the distribution's security-only origin list, disables automatic reboots by default, and uses `needrestart` for services.

Measuring and monitoring

17. How do you prove that hardening improved security?

▼ Answer

Measure with an independent tool before and after, with the same version: Lynis' hardening index, warnings and suggestions, plus functional tests from outside (password login rejected, root key login rejected, metrics port blocked, attacker banned).

18. What are the limits of a Lynis score?

▼ Answer

It is a weighted summary of checks, not a guarantee; some tests are skipped in containers; some suggestions deliberately don't apply. Compare like with like (same version, same kind of machine) and keep it as a trend with a realistic target.

19. **How would you monitor that security controls keep running?**

▼ Answer

node_exporter's systemd collector exports each service's state; an alert fires if SSH, fail2ban, the firewall, the update timer or the audit timer is not active. Lynis scores and fail2ban counters come in through the textfile collector, with alerts for a low score, a stale audit and bans.

20. **How do you get metrics from a tool that is not a Prometheus exporter?**

▼ Answer

Write a *.prom file in node_exporter's textfile directory from a script or timer, atomically (temp file + rename). Here: `lynis_report.py` and the fail2ban metrics script.

21. **How do you test Prometheus alert rules?**

▼ Answer

`promtool test rules` with synthetic input series and expected alerts at given times, including "no alert yet" inside the `for:` window. Watch out for sample spacing: samples further apart than the 5-minute lookback go stale and reset `for:` .

Testing and CI

22. **How do you test Ansible roles?**

▼ Answer

Static: `yamllint`, `ansible-lint`, syntax check. Then Molecule: create containers for several distributions, converge, idempotence, verify, destroy. Here, three distributions, and the converge/verify playbooks import the real ones.

23. **What can't you test with containers, and what do you do about it?**

▼ Answer

Anything host-wide: kernel audit, global `sysctl` settings, real boot behaviour. This project runs the baseline on the GitHub Actions runner VM, which is thrown away after the job.

24. **Why run Ansible from a container instead of installing it locally?**

▼ Answer

Pinned, identical versions of ansible-core, ansible-lint, Molecule, collections and the Docker CLI for everyone and for CI; no "works on my machine". Also, the Ansible control node does not run natively on Windows.

25. **How do you keep secrets out of a repository like this?**

▼ Answer

Generate them at runtime and keep them outside Git: the SSH key in a Docker volume, the Grafana password in a git-ignored file passed as a Docker secret, the CI admin key created per run and deleted. `.gitignore` covers `.lab/`, keys and reports. Public keys in test data are fine because they are not secret.